

Project Overview

This project is an Adaptive Multi-Agent Data Science Copilot built for the ABB assignment. It behaves like a practical AutoML assistant: the user uploads a dataset, the platform studies it, plans the machine learning workflow, trains suitable models, explains the results, and prepares an executive summary.

User Data and Metadata Considered

Dataset file	CSV, Excel, JSON, or Parquet uploaded by the user	File type, upload ID, storage path	Starts the AutoML workflow
Rows and columns	Complete tabular records	Row count and column count	Estimates dataset size
Numerical features	Integer or decimal columns	Mean, std, min, max, skewness, missing percent	Supports training and analysis
Categorical features	Text, category, boolean, or label columns	Unique values, category count, missing percent	Used for encoding and task understanding
Datetime features	Date, time, timestamp, month, or year columns	Date detection, frequency, time steps, missing timestamps	Identifies forecasting use cases
Target column	Output such as target, label, class, price, sales, churn, fraud	Suggested target, data type, number of classes	Chooses supervised learning task
Data quality	Missing values and duplicate rows	Missing percentage and duplicate count	Plans cleaning strategy
Class distribution	Distribution of output labels	Class count and imbalance ratio	Detects classification risk
Feature statistics	Behavior of selected columns	Compact feature summary	Lets agents reason without raw data
Model output	Trained model and prediction results	Best model name, metrics, model path	Supports report and download

Workflow Architecture



Architecture Summary

The system separates user experience, backend orchestration, AI reasoning, machine learning execution, explainability, and tracking. This makes the project easier to maintain and suitable for industrial workflows.

A key design decision is privacy-aware reasoning: AI agents receive metadata only. Raw data remains inside the ML pipeline for preprocessing and training.

Evaluation Criteria Covered

- Innovation: multi-agent AutoML workflow with AI reporting.
- Technical implementation: FastAPI, Next.js, CrewAI, Gemini, sklearn, XGBoost, Optuna, SHAP, MLflow, ChromaDB.
- Industrial relevance: supports repeatable, explainable, privacy-aware data science.
- Scalability: modular services, Docker setup, async jobs, WebSocket updates.

Step-by-Step Process

1. Dataset Upload and Loading

The user uploads a dataset through the dashboard. The FastAPI backend saves the file and loads it using the correct reader for CSV, Excel, JSON, or Parquet.

2. Automated Data Profiling

The DataProfiler studies the dataset with pandas and numpy. It extracts row count, column count, data types, missing values, duplicates, suggested target, class count, imbalance ratio, and time-series indicators.

3. Data Quality Analysis

The DataAgent reviews metadata and identifies practical concerns such as missing values, duplicate records, class imbalance, unsuitable feature types, or weak target selection.

4. Task Detection

The TaskAgent determines whether the problem is classification, regression, clustering, or forecasting. This decision guides model selection, preprocessing, metrics, and final reporting.

Why This Process Is Useful

- It reduces manual effort for users who may not know every ML step.
- It keeps the workflow consistent across different datasets.
- It allows AI agents to guide decisions while proven ML libraries perform computation.
- It protects data privacy by passing only metadata to the LLM layer.

Modeling, Evaluation, and Reporting

5. Workflow Planning

The WorkflowAgent prepares preprocessing actions such as missing value handling, categorical encoding, numerical scaling, target preparation, and model-ready feature creation.

6. Model Selection

The ModelStrategistAgent selects algorithms based on the detected task. Classification uses Logistic Regression, Random Forest, and XGBoost. Regression uses Linear Regression, Random Forest Regressor, and XGBoost Regressor. Clustering uses KMeans and DBSCAN. Forecasting uses ARIMA and Exponential Smoothing.

7. Training and Optimization

The ML engine trains multiple models using scikit-learn, XGBoost, and statsmodels. Optuna is used for hyperparameter optimization so the system can improve performance beyond default settings.

8. Evaluation

The EvaluationAgent benchmarks models with task-specific metrics. Classification uses accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix. Regression uses RMSE, MAE, and R2 score. Clustering uses silhouette score. Forecasting uses AIC, BIC, and forecast values.

9. Explainability and Final Report

The ExplainabilityAgent uses SHAP to show feature importance and model behavior. The ReportAgent generates a business-friendly summary with data findings, model comparison, best model, explainability insights, and recommendations.

Classification	Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion Matrix
Regression	RMSE, MAE, R2 Score
Clustering	Number of Clusters, Silhouette Score
Forecasting	AIC, BIC, Forecast Values

Expected Output of the Project

The expected output is a complete automated data science workflow presented through a professional dashboard. The user receives model results, performance comparisons, explainability insights, and a downloadable trained model.

1. Interactive Dashboard

The dashboard allows dataset upload and real-time monitoring of profiling, task detection, preprocessing, training, optimization, evaluation, explainability, and report generation.

2. Automated ML Results

The system produces a dataset summary, data quality analysis, detected task type, preprocessing workflow, selected models, tuning results, benchmarking, and best model selection.

3. Explainable AI Insights

SHAP-based feature importance and AI-generated interpretation help the user understand which features influence predictions.

4. Exportable Best Model

The best-performing model is saved as a downloadable .pkl file so it can be reused, tested, or integrated into another application.

5. Executive AI Report

The final report explains the dataset overview, ML task, data quality findings, model comparison, best model recommendation, explainability insights, and conclusion.

Scalability and Robustness

The solution is modular: frontend, backend, agents, ML engine, tracking, and memory are separated. The backend uses asynchronous execution and WebSocket updates for long-running jobs. Docker Compose supports consistent deployment. MLflow tracks experiments, while ChromaDB stores reusable workflow memory.

Conclusion

The Adaptive Multi-Agent Data Science Copilot combines originality, practical technical implementation, industrial usefulness, and scalable design. It reduces manual machine learning effort while keeping the workflow explainable, traceable, and privacy-aware.